

APLICAȚIA 3: IMPLEMENTAREA CIRCUITELOR COMBINAȚIONALE ÎN LIMBAJUL VERILOG HDL

Universitatea Politehnica Timișoara
Facultatea de Automatica si Calculatoare
Departamentul de Calculatoare și Tehnologia Informației

Martie 2025

1 Rezumat

Acest laborator își propune prezentarea succintă a conceptelor de Verilog HDL necesare descrierii circuitelor combinaționale.

Vor fi trecute în revistă: tipurile de modele de descriere (flux de date, comportamental, structural, mixt) specifice unui circuit cu exemplificare pentru celula de însumare pe 1 bit, modulul Verilog HDL.

Circuitul descris este o celulă de însumare pe 1 bit cu 3 intrări. Se va folosi ModelSim pentru simulare și Quartus pentru sinteză, post-sinteză și analiză de timp.

2 Obiectivele lucrării

Obiectivul acestui laborator este acela de cunoaștere a noțiunilor fundamentale de Verilog HDL necesare descrierii unui circuit combinațional. De asemenea, folosirea acestora în vederea realizării unui design simplu: celula de însumare pe un bit.

2.1 Obiective tehnice

- Familiarizare cu placa Terasic DE10-Lite.
- Realizarea unui design simplu: celulă sumator pe 1 bit folosind Verilog HDL, simularea lui folosind ModelSim.
- Sinteză și implementare design-ului folosind Quartus.
- Configurare FPGA MAX 10 de pe placa DE10-Lite.

Timp necesar: 2-3 ore

3 Pregătirea pentru laborator

- Citiți documentul înainte de a începe realizarea practică.
- Salvați output-urile pentru fiecare cerință sau anunțați cadrul didactic în vederea prezentării rezultatelor.

4 Echipamente și Materiale

Necesar	Cantitate
Software Quartus Prime Lite Edition, cu suport pentru dispozitivul MAX10	1
Software ModelSim-Intel FPGA Standard Edition	1
Placă Terasic DE10-Lite	1

Table 1: Echipamente și materiale necesare

5 Scurtă introducere în limbajul VerilogHDL

Verilog este un limbaj de descriere hardware (HDL) care permite modelarea unui sistem digital la diferite niveluri de abstractizare: algoritmic, comportamental, poartă logică, sau chiar la nivelul comutator. Acesta este utilizat datorită simplității sale, permițând proiectarea și verificarea rapidă a circuitelor.

5.1 Caracteristici principale ale Verilog HDL

- Definirea de primitive logice elementare pentru porțile logice.
- Permite modelarea circuitelor sub formă de flux de date, comportamental, structural și mixt.
- Modelarea execuției concurente și a timpului.

Modelarea unui sistem digital în Verilog se poate realiza prin:

- Modelarea prin flux de date.
- Modelarea comportamentală.
- Modelarea structurală.
- Modelarea mixtă.

6 Modelarea prin flux de date

Aceasta se bazează pe instrucțiunea `assign`, ce realizează evaluarea continuă a expresiei din dreapta egalului, atribuind noua valoare unui semnal.

Listing 1: Modelare prin flux de date a unui sumator complet pe 1 bit

```
module full_adder_cell
  (input a, b, c_in,
   output s, c_out);

  assign #2 c_out = (a & b) | (a & c_in) | (b & c_in);
  assign #2 s = a ^ b ^ c_in;

endmodule
```

7 Modelarea comportamentală

Aceasta se bazează pe construcții de tip `always`, permițând descrierea de nivel înalt a comportamentului unui circuit hardware.

Listing 2: Modelare comportamentală a unui sumator complet pe 1 bit

```
module full_adder_cell
  (input a, b, c_in,
   output reg s, c_out);
```

```

always @(a, b, c_in)
begin
    s = a ^ b ^ c_in;
    c_out = (a & b) | (a & c_in) | (b & c_in);
end

endmodule

```

8 Modelarea structurală

Modelarea structurală utilizează instanțierea modulelor în interiorul unui modul principal, interconectate prin semnale de tip `wire`. Figura 1 prezintă structura unui sumator pe 1 bit.

Listing 3: Modelare structurală a unui sumator complet pe 1 bit

```

module full_adder_cell
(input a, b, c_in,
output reg s, c_out);

wire w1, w2, w3, w4;

and a1(w1, a, b);
and a2(w2, a, c_in);
and a3(w3, b, c_in);
or o1(c_out, w1, w2, w3);
xor x1(w4, a, b);
xor x2(s, w4, c_in);
endmodule

```

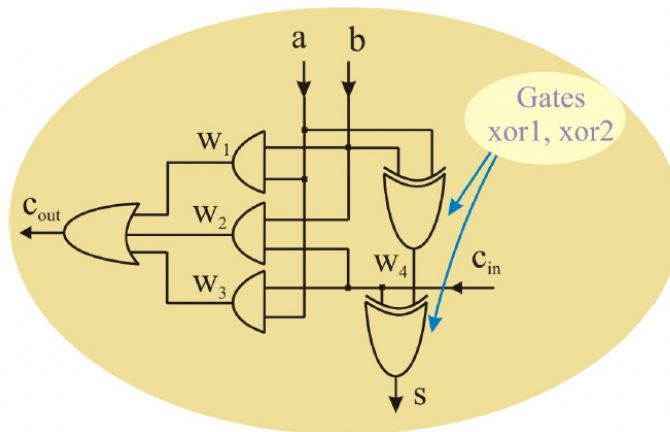


Figure 1: Schema sumatorului complet pe 1 bit utilizând modelarea structurală

9 Implementarea unui circuit sumator pe 2 biți

Pentru implementarea unui circuit sumator pe 2 biți pe un dispozitiv, sunt necesari mai mulți pași esențiali.

9.1 Descrierea si simularea design-ului

Circuitul va fi descris în Verilog HDL. Se va folosi descrierea comportamentală sau flux de date pentru celula sumator complet pe 1 bit, respectiv descriere structurală pentru sumatorul pe 2 biți alcătuit din două celule sumator complet.

- **adder_1bit_cell** – aceasta reprezintă descrierea celulei sumator complet pe 1 bit; ea va avea în interfață 3 semnale de intrare (a , b , c_in) și 2 semnale de ieșire (s , c_out); descrierea acestui modul este una de tip flux de date sau comportamental.
- **adder_2bits** – este modulul care va implementa circuitul sumator pe 2 biți; descrierea acestuia va fi structurală. Se vor folosi 2 instanțe de celule sumator complet pe 1 bit. Mai jos este descrisă interfața unui astfel de sumator.

Listing 4: Modelare structurală a unui sumator complet pe 2 biți

```
module adder_2bits
  (input c_in,
   input [1:0] a, b;
   output [1:0] s,
   output c_out);

  // DE COMPLETAT LISTA DE SEMNALE INTERNE

  // DE COMPLETAT DESCRIEREA STRUCTURALA

endmodule
```

În figura 2 următoare se prezintă structura de foldere și fișiere cu care vom lucra. Ne reamintim pentru a utiliza Model Sim vom modifica fisierul testbench.v respectiv fisierul wave.so (unde adauga formele de undă ce dorim să le monitorizăm). Fisierele de modificat pentru simulare sunt marcate cu galben.

Listing 5: Testbench pentru un sumator complet pe 2 biți

```
module adder_2bits_tb;

  // Inputs
  reg [1:0] a; // operand 1
  reg [1:0] b; // operand 2

  // Outputs
  wire [1:0] sum;
  wire cout;

  // Instantiate the Unit Under Test (UUT)
  adder_2bits uut (
    .a(a),
    .b(b),
```

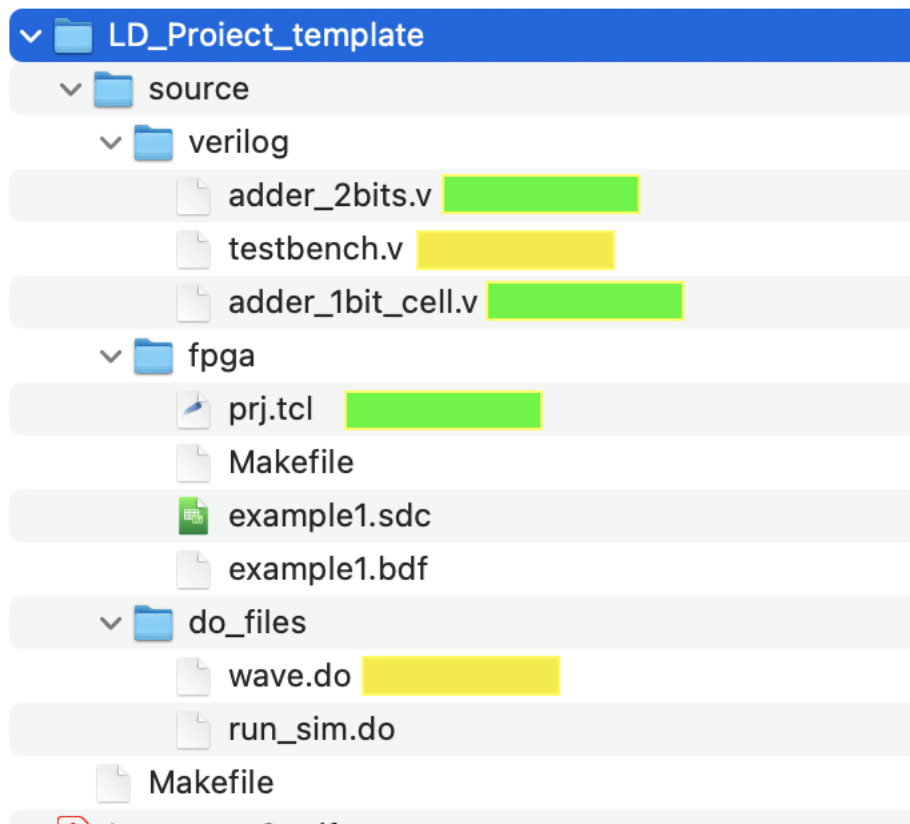


Figure 2: Structura folderelor cu care vom lucra.

```

        .c_in(1'b0),
        .s(sum),
        .c_out(cout)
    );

initial begin
    // Initialize Inputs
    a = 0;
    b = 0;

    // Wait 100 ns for global reset to finish
    #100;
end

always // toggle inputs for two-bit adder
begin
    #25 a = a + 1'b1;
    #50 b = b + 1'b1;
end

endmodule

```

Se va simula circuitul folosind simulatorul Model Sim cum este detaliat în lucrarea precedentă.

Se modifica fisierul wave.so prin adaugarea semnalelor ce urmeaza a fi monitorizare, de regula intrarile si iesirile. Utilizand comanda cd ne pozitionam in directorul proiectului acolo unde avem fisierul makefile, si din terminal rulam make. Prin aceasta comanda se va rula vsim ce va deschide ModelSim si va deschide o simulare care va contine formele de unda din fisierul wave.so modificat.

9.2 Sinteza și implementarea circuitului

Vom folosi Quartus pentru sinteză și implementare și vom lucra în modul “linie de comandă” (script).

Design-ul care trebuie implementat pe placa FPGA este `adder_2bits.v`. Ca și modul component va trebui să regasim si fisierul `adder_1bit_cell.v`.

Toate intrările și ieșirile design-ului trebuie asigurate către pinii corespunzători:

- **Semnalele de intrare:** cin va fi conectat la 0, respectiv `[1:0]a`, `[1:0]b` pe care le vom conecta la întrerupătoarele `SW0`, `SW1`, `SW2`, `SW3`, care sunt conectate la FPGA la pinii `PIN_C10`, `PIN_C11`, `PIN_D12`, `PIN_C12`.
- **Semnalele de ieșire:** cout și `[1:0]s`, le vom conecta la LED-urile `LEDR0`, `LEDR1`, `LEDR2`, conectat la FPGA la pinii `PIN_A8`, `PIN_A9`, `PIN_A10`.

Urmează să configurăm pinii fixigi ai FPGA-ului prin fișierul `prj.tcl` care va avea următorul conținut:

Listing 6: maparea semnalelor pentru un sumator complet pe 2 biti

```
project_new example1 -overwrite
set_global_assignment -name FAMILY MAX10
set_global_assignment -name DEVICE 10M50DAF484C7G
set_global_assignment -name VERILOG_FILE adder_1bit_cell.v
set_global_assignment -name VERILOG_FILE adder_2bits.v
set_global_assignment -name TOP_LEVEL_ENTITY adder_2bits
set_location_assignment -to a[0] PIN_C10
set_location_assignment -to a[1] PIN_C11
// de completat cu restul intrarilor/iesirilor
set_location_assignment -to cout PIN_A9
load_package flow
execute_flow -compile
project_close
```

Acest fișier reprezintă script-ul folosit de Quartus pentru rularea compilării design-ului și generarea fișierului de programare/configurare (`.sof`).

Apoi, din terminalul Linux vom naviga în directorul fpga unde se regaseste fisierul `prj.tcl` respectiv `makefile`:

```
cd nume_director
```

ruland comanda make si conectand cablul USB la placa DE10-Lite se executa urmatoarele comenzi:

```
quartus_sh -t prj.tcl
quartus_pgm -l
quartus_pgm --auto
quartus_pgm -m JTAG -o "p;example1.sof@1"
```

Dacă au fost întâmpinate erori, programul le va raporta. Dacă nu, iar compilarea a avut succes, fișierul de programare va fi prezent în directorul curent. Pentru a verifica existența fișierului de programare (`.sof`) executăm comanda:

```
ls -al
```

sau

```
ls -al *.sof
```

10 Concluzii

Acest laborator introduce studenții în utilizarea Verilog HDL pentru modelarea și implementarea circuitelor combinaționale. Prin parcurgerea exercițiilor propuse, studenții își vor consolida cunoștințele în proiectarea circuitelor digitale și utilizarea FPGA-urilor.

11 Exerciții

Realizați pașii indicați. Completați liniile de cod lipsă. Realizați design-ul și verificați funcționarea corectă a design-ului pe placă!